Práctica: API de Películas (Backend) + Consumo desde Frontend

Modalidad: Trabajo en parejas o grupos separados — un equipo construye el **Backend** y otro el **Frontend**, y al final se integran.

Patrón usado en el backend: MVC (Modelo - Vista* - Controlador). *Como es una API REST, la "Vista" es el JSON que se devuelve al cliente.

Recurso: `peliculas`

🎯 Objetivos de aprendizaje

Al terminar esta práctica, el estudiante será capaz de:

- Construir una API REST con Node.js, Express y MongoDB (Mongoose) organizada en capas (MVC).
- Implementar operaciones CRUD completas (Crear, Leer, Actualizar, Eliminar).
- Consumir esa API desde un frontend, usando **React** o **JavaScript Vanilla**, a elección del equipo.
- Coordinarse con otro equipo a partir de un contrato de API (rutas y formato de datos) sin necesidad de ver el código del otro lado.

🧩 Organización sugerida

Equipo	Responsabilidad
Equipo Backend	Construye la API REST completa (Parte 1)
Equipo Frontend	Construye la interfaz que consume la API (Parte 2, opción React o Vanilla JS)

> Los equipos pueden trabajar en paralelo. El Frontend puede simular datos al inicio (o usar la API ya desplegada por el Backend) y luego conectar todo en la fase de integración final (Parte 3).

📋 Requisitos previos

- Node.js instalado (v18 o superior).
- MongoDB local corriendo, o una base de datos en **MongoDB Atlas** (recomendado para que ambos equipos usen la misma base).
- Un cliente para probar la API: Postman, Thunder Client (extensión de VS Code) o Insomnia.
- Editor de código (VS Code recomendado).

🔧 PARTE 1 — Backend: API REST con Node, Express y Mongoose (Patrón MVC)

Contrato de la API (esto es lo que el equipo de Frontend necesita saber)

Método	Ruta	Descripción	Body (JSON)
GET	`/api/peliculas`	Lista todas las películas	—
GET	`/api/peliculas/:id`	Obtiene una película por id	—
POST	`/api/peliculas`	Crea una película	`{ titulo, genero, anio, director, calificacion }`
PUT	`/api/peliculas/:id`	Actualiza una película	mismos campos, todos opcionales
DELETE	`/api/peliculas/:id`	Elimina una película	—

Respuesta estándar de éxito:

```json
{
  "ok": true,
  "data": { ... }
}
```

Respuesta estándar de error:

```json
{
  "ok": false,
  "mensaje": "Descripción del error"
}
```

Paso 1.1 — Inicializar el proyecto

```bash
```

```
mkdir api-peliculas
cd api-peliculas
npm init -y
npm install express mongoose cors dotenv
npm install --save-dev nodemon
```

En `package.json`, agrega el script de desarrollo:

```json
"scripts": {
  "dev": "nodemon src/server.js",
  "start": "node src/server.js"
}
```

Paso 1.2 — Estructura de carpetas (MVC)

```
api-peliculas/
├── src/
│   ├── config/
│   │   └── db.js
│   ├── models/
│   │   └── Pelicula.js
│   ├── controllers/
│   │   └── peliculas.controller.js
│   ├── routes/
│   │   └── peliculas.routes.js
│   ├── app.js
│   └── server.js
├── .env
└── package.json
```

Paso 1.3 — Variables de entorno

Crea el archivo `.env`:

```
PORT=4000
MONGO_URI=mongodb://127.0.0.1:27017/db_peliculas
```

> Si usan MongoDB Atlas, reemplacen `MONGO_URI` por la cadena de conexión que provee Atlas.

Paso 1.4 — Conexión a la base de datos (`src/config/db.js`)

```js
const mongoose = require('mongoose');

const conectarDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('✅ MongoDB conectado');
  } catch (error) {
    console.error('❌ Error al conectar a MongoDB:', error.message);
    process.exit(1);
  }
};

module.exports = conectarDB;
```

Paso 1.5 — Modelo (M de MVC) — `src/models/Pelicula.js`

```js
```

```
const mongoose = require('mongoose');

const peliculaSchema = new mongoose.Schema(
  {
    titulo: {
      type: String,
      required: [true, 'El título es obligatorio'],
      trim: true,
    },
    genero: {
      type: String,
      required: [true, 'El género es obligatorio'],
    },
    anio: {
      type: Number,
      required: [true, 'El año es obligatorio'],
    },
    director: {
      type: String,
      required: [true, 'El director es obligatorio'],
    },
    calificacion: {
      type: Number,
      min: 0,
      max: 10,
      default: 0,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Película', peliculaSchema);
```

Paso 1.6 — Controlador (C de MVC) — `src/controllers/peliculas.controller.js`

Aquí va la lógica de negocio. El controlador nunca habla directamente de rutas, solo recibe `req` y `res`.

```
js
```

```
const Pelicula = require('../models/Pelicula');

// GET /api/peliculas
const obtenerPeliculas = async (req, res) => {
  try {
    const peliculas = await Pelicula.find();
    res.status(200).json({ ok: true, data: peliculas });
  } catch (error) {
    res.status(500).json({ ok: false, mensaje: 'Error al obtener las películas'
  }
};

// GET /api/peliculas/:id
const obtenerPeliculaPorId = async (req, res) => {
  try {
    const pelicula = await Pelicula.findById(req.params.id);
    if (!pelicula) {
      return res.status(404).json({ ok: false, mensaje: 'Película no encontrada
    }
    res.status(200).json({ ok: true, data: pelicula });
  } catch (error) {
    res.status(500).json({ ok: false, mensaje: 'Error al obtener la película' }
  }
};

// POST /api/peliculas
const crearPelicula = async (req, res) => {
  try {
    const nuevaPelicula = new Pelicula(req.body);
    const peliculaGuardada = await nuevaPelicula.save();
    res.status(201).json({ ok: true, data: peliculaGuardada });
  } catch (error) {
    res.status(400).json({ ok: false, mensaje: error.message });
  }
};

// PUT /api/peliculas/:id
const actualizarPelicula = async (req, res) => {
  try {
    const peliculaActualizada = await Pelicula.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
```

```
    );
    if (!peliculaActualizada) {
      return res.status(404).json({ ok: false, mensaje: 'Película no encontrada
    }
    res.status(200).json({ ok: true, data: peliculaActualizada });
  } catch (error) {
    res.status(400).json({ ok: false, mensaje: error.message });
  }
};

// DELETE /api/peliculas/:id
const eliminarPelicula = async (req, res) => {
  try {
    const peliculaEliminada = await Pelicula.findByIdAndDelete(req.params.id);
    if (!peliculaEliminada) {
      return res.status(404).json({ ok: false, mensaje: 'Película no encontrada
    }
    res.status(200).json({ ok: true, data: peliculaEliminada });
  } catch (error) {
    res.status(500).json({ ok: false, mensaje: 'Error al eliminar la película'
  }
};

module.exports = {
  obtenerPeliculas,
  obtenerPeliculaPorId,
  crearPelicula,
  actualizarPelicula,
  eliminarPelicula,
};
```

Paso 1.7 — Rutas — `src/routes/peliculas.routes.js`

```
js
```

```
const express = require('express');
const router = express.Router();
const {
  obtenerPeliculas,
  obtenerPeliculaPorId,
  crearPelicula,
  actualizarPelicula,
  eliminarPelicula,
} = require('../controllers/peliculas.controller');

router.get('/', obtenerPeliculas);
router.get('/:id', obtenerPeliculaPorId);
router.post('/', crearPelicula);
router.put('/:id', actualizarPelicula);
router.delete('/:id', eliminarPelicula);

module.exports = router;
```

Paso 1.8 — App de Express — `src/app.js`

js

```
const express = require('express');
const cors = require('cors');
const peliculasRoutes = require('./routes/peliculas.routes');

const app = express();

app.use(cors());
app.use(express.json());

app.use('/api/peliculas', peliculasRoutes);

// Ruta de prueba
app.get('/', (req, res) => {
  res.send('API de Películas funcionando 🎬');
});

module.exports = app;
```

Paso 1.9 — Servidor — `src/server.js`

```js
require('dotenv').config();
const app = require('./app');
const conectarDB = require('./config/db');

const PORT = process.env.PORT || 4000;

conectarDB().then(() => {
  app.listen(PORT, () => {
    console.log(`🚀 Servidor corriendo en http://localhost:${PORT}`);
  });
});
```

Paso 1.10 — Probar la API

```bash
npm run dev
```

Con Postman/Thunder Client, probar en este orden:

1. `POST http://localhost:4000/api/peliculas` con body:

```json
{ "titulo": "Interstellar", "genero": "Ciencia ficción", "anio": 2014, "dire
```

2. `GET http://localhost:4000/api/peliculas` → debe listar la película creada.
3. `GET http://localhost:4000/api/peliculas/:id` con el `_id` recibido.
4. `PUT http://localhost:4000/api/peliculas/:id` cambiando la `calificacion`.
5. `DELETE http://localhost:4000/api/peliculas/:id`.

✅ **Checkpoint del equipo Backend:** las 5 operaciones responden con el formato `{ ok, data }` o `{ ok, mensaje }`, y `cors()` está habilitado para que el Frontend pueda consumir la API desde otro puerto.

🎨 PARTE 2 — Frontend: consumir la API

El equipo de Frontend elige **una** de las dos opciones. Ambas construyen lo mismo: una pantalla que **lista, crea, edita y elimina** películas.

> Si el Backend aún no está listo, pueden apuntar a `http://localhost:4000/api/peliculas` y probar en cuanto el otro equipo levante el servidor.

Opción A — Vanilla JavaScript paso a paso

Paso 2A.1 — Estructura

```
frontend-vanilla/
├── index.html
├── style.css
└── script.js
```

Paso 2A.2 — `index.html`

html

```html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <title>Películas</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <h1>🎬 Catálogo de Películas</h1>

  <form id="form-pelicula">
    <input type="hidden" id="pelicula-id" />
    <input type="text" id="titulo" placeholder="Título" required />
    <input type="text" id="genero" placeholder="Género" required />
    <input type="number" id="anio" placeholder="Año" required />
    <input type="text" id="director" placeholder="Director" required />
    <input type="number" id="calificacion" placeholder="Calificación" min="0" m
    <button type="submit">Guardar</button>
  </form>

  <ul id="lista-peliculas"></ul>

  <script src="script.js"></script>
</body>
</html>
```

Paso 2A.3 — `script.js`

```
js
```

```
const API_URL = 'http://localhost:4000/api/peliculas';

const form = document.getElementById('form-pelicula');
const lista = document.getElementById('lista-peliculas');

// Leer todas las películas
const obtenerPeliculas = async () => {
  const respuesta = await fetch(API_URL);
  const resultado = await respuesta.json();
  renderizar(resultado.data);
};

// Pintar la lista en el HTML
const renderizar = (peliculas) => {
  lista.innerHTML = '';
  peliculas.forEach((pelicula) => {
    const li = document.createElement('li');
    li.innerHTML = `
      <strong>${pelicula.titulo}</strong> (${pelicula.anio}) - ${pelicula.gener
      <br>Director: ${pelicula.director} | Calificación: ${pelicula.calificacio
      <button onclick="editarPelicula('${pelicula._id}')">Editar</button>
      <button onclick="eliminarPelicula('${pelicula._id}')">Eliminar</button>
    `;
    lista.appendChild(li);
  });
};

// Crear o actualizar (según si hay id)
form.addEventListener('submit', async (evento) => {
  evento.preventDefault();

  const id = document.getElementById('pelicula-id').value;
  const datos = {
    titulo: document.getElementById('titulo').value,
    genero: document.getElementById('genero').value,
    anio: Number(document.getElementById('anio').value),
    director: document.getElementById('director').value,
    calificacion: Number(document.getElementById('calificacion').value),
  };

  if (id) {
    await fetch(`${API_URL}/${id}`, {
      method: 'PUT',
```

```
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(datos),
    });
  } else {
    await fetch(API_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(datos),
    });
  }

  form.reset();
  document.getElementById('pelicula-id').value = '';
  obtenerPeliculas();
});

// Cargar datos de una película en el formulario para editar
window.editarPelicula = async (id) => {
  const respuesta = await fetch(`${API_URL}/${id}`);
  const resultado = await respuesta.json();
  const pelicula = resultado.data;

  document.getElementById('pelicula-id').value = pelicula._id;
  document.getElementById('titulo').value = pelicula.titulo;
  document.getElementById('genero').value = pelicula.genero;
  document.getElementById('anio').value = pelicula.anio;
  document.getElementById('director').value = pelicula.director;
  document.getElementById('calificacion').value = pelicula.calificacion;
};

// Eliminar
window.eliminarPelicula = async (id) => {
  await fetch(`${API_URL}/${id}`, { method: 'DELETE' });
  obtenerPeliculas();
};

// Carga inicial
obtenerPeliculas();
```

Paso 2A.4 — Probar

Abrir `index.html` con **Live Server** (extensión de VS Code) para evitar problemas de CORS al abrir el archivo directamente.

✅ **Checkpoint Vanilla JS:** listar, crear, editar y eliminar funcionan contra la API real.

Opción B — React paso a paso

Paso 2B.1 — Crear el proyecto

bash

```
npm create vite@latest frontend-react -- --template react
cd frontend-react
npm install
npm install axios
```

Paso 2B.2 — Estructura

```
frontend-react/
└── src/
    ├── services/
    │   └── peliculasService.js
    ├── components/
    │   ├── FormularioPelicula.jsx
    │   └── ListaPeliculas.jsx
    ├── App.jsx
    └── main.jsx
```

Paso 2B.3 — Servicio (capa que habla con la API) — `src/services/peliculasService.js`

js

```
import axios from 'axios';

const API_URL = 'http://localhost:4000/api/peliculas';

export const obtenerPeliculas = () => axios.get(API_URL);
export const obtenerPeliculaPorId = (id) => axios.get(`${API_URL}/${id}`);
export const crearPelicula = (datos) => axios.post(API_URL, datos);
export const actualizarPelicula = (id, datos) => axios.put(`${API_URL}/${id}`,
export const eliminarPelicula = (id) => axios.delete(`${API_URL}/${id}`);
```

Paso 2B.4 — Componente de formulario — `src/components/FormularioPelicula.jsx`

jsx

```jsx
import { useState, useEffect } from 'react';

const valoresIniciales = {
  titulo: '', genero: '', anio: '', director: '', calificacion: '',
};

function FormularioPelicula({ peliculaSeleccionada, onGuardar }) {
  const [form, setForm] = useState(valoresIniciales);

  useEffect(() => {
    if (peliculaSeleccionada) {
      setForm(peliculaSeleccionada);
    } else {
      setForm(valoresIniciales);
    }
  }, [peliculaSeleccionada]);

  const manejarCambio = (evento) => {
    setForm({ ...form, [evento.target.name]: evento.target.value });
  };

  const manejarEnvio = (evento) => {
    evento.preventDefault();
    onGuardar({
      ...form,
      anio: Number(form.anio),
      calificacion: Number(form.calificacion),
    });
    setForm(valoresIniciales);
  };

  return (
    <form onSubmit={manejarEnvio}>
      <input name="titulo" placeholder="Título" value={form.titulo} onChange={m
      <input name="genero" placeholder="Género" value={form.genero} onChange={m
      <input name="anio" type="number" placeholder="Año" value={form.anio} onCh
      <input name="director" placeholder="Director" value={form.director} onCha
      <input name="calificacion" type="number" placeholder="Calificación" value
      <button type="submit">Guardar</button>
    </form>
  );
}
```

```
export default FormularioPelicula;
```

Paso 2B.5 — Componente de lista — `src/components/ListaPeliculas.jsx`

jsx

```jsx
function ListaPeliculas({ peliculas, onEditar, onEliminar }) {
  return (
    <ul>
      {peliculas.map((pelicula) => (
        <li key={pelicula._id}>
          <strong>{pelicula.titulo}</strong> ({pelicula.anio}) - {pelicula.gene
          <br />
          Director: {pelicula.director} | Calificación: {pelicula.calificacion}
          <br />
          <button onClick={() => onEditar(pelicula)}>Editar</button>
          <button onClick={() => onEliminar(pelicula._id)}>Eliminar</button>
        </li>
      ))}
    </ul>
  );
}

export default ListaPeliculas;
```

Paso 2B.6 — `App.jsx` (componente principal)

jsx

```jsx
import { useState, useEffect } from 'react';
import FormularioPelicula from './components/FormularioPelicula';
import ListaPeliculas from './components/ListaPeliculas';
import {
  obtenerPeliculas,
  crearPelicula,
  actualizarPelicula,
  eliminarPelicula,
} from './services/peliculasService';

function App() {
  const [peliculas, setPeliculas] = useState([]);
  const [peliculaSeleccionada, setPeliculaSeleccionada] = useState(null);

  const cargarPeliculas = async () => {
    const respuesta = await obtenerPeliculas();
    setPeliculas(respuesta.data.data);
  };

  useEffect(() => {
    cargarPeliculas();
  }, []);

  const manejarGuardar = async (datos) => {
    if (peliculaSeleccionada) {
      await actualizarPelicula(peliculaSeleccionada._id, datos);
    } else {
      await crearPelicula(datos);
    }
    setPeliculaSeleccionada(null);
    cargarPeliculas();
  };

  const manejarEliminar = async (id) => {
    await eliminarPelicula(id);
    cargarPeliculas();
  };

  return (
    <div>
      <h1>🎬 Catálogo de Películas</h1>
      <FormularioPelicula
        peliculaSeleccionada={peliculaSeleccionada}
```

```
        onGuardar={manejarGuardar}
      />
      <ListaPeliculas
        peliculas={peliculas}
        onEditar={setPeliculaSeleccionada}
        onEliminar={manejarEliminar}
      />
    </div>
  );
}

export default App;
```

Paso 2B.7 — Ejecutar

```
bash

npm run dev
```

✅ **Checkpoint React:** el formulario crea y edita según si hay una película seleccionada; la lista se actualiza automáticamente tras cada acción.

🔗 PARTE 3 — Integración final entre equipos

1. El equipo Backend comparte la URL base de su API (ej. `http://localhost:4000/api/peliculas`) y confirma que `cors()` está activo.
2. El equipo Frontend reemplaza cualquier URL de prueba por la URL real del Backend.
3. Prueban juntos el flujo completo: crear una película desde el Frontend → verificarla en la base de datos (Compass o Atlas) → editarla → eliminarla.
4. Si algo falla, revisan en este orden: consola del navegador (frontend) → consola del servidor (backend) → nombres exactos de los campos del `body` (deben coincidir con el modelo Mongoose).

📦 Entregables

- Repositorio del Backend con la estructura MVC completa.
- Repositorio del Frontend (React o Vanilla JS) consumiendo la API real.
- Captura o video corto mostrando el CRUD funcionando de punta a punta.

✅ Rúbrica sugerida

Criterio	Puntos
Estructura MVC correcta en el backend	20
CRUD completo y funcional (backend)	30
Frontend consume correctamente la API (los 4 métodos)	30
Manejo de errores (mensajes claros al usuario)	10
Integración exitosa entre ambos equipos	10